YOPY — Game Development Structure

Plan de développement — 18 jeux × 2 couches × 10 sprints

Architecture frontend (PHP) + backend (PHP) + collecte signaux IA

Jeux total	18 jeux répartis en 5 catégories
Sprints	10 sprints de 1 semaine (Phases 1–9 + Intégration IA)
Frontend	`PRO-PFE-YOPY\yopy-platform\views\child\games\{game}.php`
Backend	`PRO-PFE-YOPY\yopy-platform\games\{game}\{game}_backend.php`
Signaux IA	26 signaux comportementaux collectés en session PHP uniquement
États IA	ENGAGÉ | FRUSTRÉ | CONFUS | SATISFAIT

1. Structure de fichiers — Vue d'ensemble

Chaque jeu possède exactement deux fichiers principaux :

Frontend PHP : `views/child/games/{game_id}.php` — Interface HTML + CSS + JS du jeu
Backend PHP : `games/{game_id}/{game_id}_backend.php` — API JSON : session, score, validation

```
PRO-PFE-YOPY/yopy-platform/
│
├── views/child/games/                    ← FRONTEND (18 fichiers PHP)
│   ├── memory_matrix.php
│   ├── pattern_builder.php
│   ├── mini_sudoku.php
│   ├── logic_gates.php
│   ├── tap_the_light.php
│   ├── color_switch.php
│   ├── avoid_obstacles.php
│   ├── quick_math.php
│   ├── word_builder.php
│   ├── science_quiz.php
│   ├── shape_geometry.php
│   ├── block_fit.php
│   ├── maze_escape.php
│   ├── jigsaw_puzzle.php
│   ├── bridge_builder.php
│   ├── mini_quest.php
│   ├── treasure_hunt.php
│   └── emotion_island.php
│
├── games/                                ← BACKEND (18 dossiers)
│   ├── memory_matrix/
│   │   └── memory_matrix_backend.php
│   ├── pattern_builder/
│   │   └── pattern_builder_backend.php
│   ├── mini_sudoku/
```

```
│   │   └── mini_sudoku_backend.php
│   ├── logic_gates/
│   │   └── logic_gates_backend.php
│   ├── tap_the_light/
│   │   └── tap_the_light_backend.php
│   ├── color_switch/
│   │   └── color_switch_backend.php
│   ├── avoid_obstacles/
│   │   └── avoid_obstacles_backend.php
│   ├── quick_math/
│   │   └── quick_math_backend.php
│   ├── word_builder/
│   │   └── word_builder_backend.php
│   ├── science_quiz/
│   │   └── science_quiz_backend.php
│   ├── shape_geometry/
│   │   └── shape_geometry_backend.php
│   ├── block_fit/
│   │   └── block_fit_backend.php
│   ├── maze_escape/
│   │   └── maze_escape_backend.php
│   ├── jigsaw_puzzle/
│   │   └── jigsaw_puzzle_backend.php
│   ├── bridge_builder/
│   │   └── bridge_builder_backend.php
│   ├── mini_quest/
│   │   └── mini_quest_backend.php
│   ├── treasure_hunt/
│   │   └── treasure_hunt_backend.php
│   └── emotion_island/
│       └── emotion_island_backend.php
│
└── public/js/games/                        ← JS ENGINES (18 fichiers)
    ├── GameEngine.js                        ← Base class (partagée)
    ├── BehaviorCollector.js                 ← Collecte signaux IA
    ├── memory_matrix.js
    ├── pattern_builder.js
    ├── ... (un fichier JS par jeu)
    └── emotion_island.js
```

2. Tableau complet — Jeux × Fichiers × Signaux IA

□ Cognitive & Logic

Jeu	Sprint	Frontend PHP	Backend PHP
□ **Memory Matrix** *Grid-based recall game. Progressive difficulty, time decay mechanics.*	**W1**	…/games/memory_matrix.php	… /memory_matrix /memory_matrix_backend.php
□ **Pattern Builder** *Complete sequences: color, shape, logic rules.*	**W1**	…/games/pattern_builder.php	… /pattern_builder /pattern_builder_backend.php

□ **Mini Sudoku** *Simplified NxN constraint satisfaction problem for kids.*	**W2**	…/games/mini_sudoku.php	… /mini_sudoku /mini_sudoku_backend.php
⚡ **Logic Gates Puzzle** *Simulate AND, OR, NOT gates visually. CS-inspired.*	**W2**	…/games/logic_gates.php	… /logic_gates /logic_gates_backend.php

⚡ Reflex & Reaction

Jeu	Sprint	Frontend PHP	Backend PHP
□ **Tap the Light** *Tap appearing objects before timeout. Reaction time metric.*	**W3**	…/games/tap_the_light.php	… /tap_the_light /tap_the_light_backend.php
□ **Color Switch Rush** *React only to target color. Inhibitory control training.*	**W3**	…/games/color_switch.php	… /color_switch /color_switch_backend.php
□ **Avoid the Obstacles** *Endless runner with increasing velocity.*	**W4**	…/games/avoid_obstacles.php	… /avoid_obstacles /avoid_obstacles_backend.php

□ Educational

Jeu	Sprint	Frontend PHP	Backend PHP
➕**Quick Math Duel** *Timed arithmetic challenges with adaptive difficulty.*	**W4**	…/games/quick_math.php	… /quick_math /quick_math_backend.php
□ **Word Builder** *Form words from letters. Dictionary validation.*	**W5**	…/games/word_builder.php	… /word_builder /word_builder_backend.php
□ **Science Quiz Adventure** *Quiz + exploration system across science topics.*	**W5**	…/games/science_quiz.php	… /science_quiz /science_quiz_backend.php
□ **Shape & Geometry Game**	**W5**	…/games/shape_geometry.php	… /shape_geometry /shape_geometry_backend.php

Identify shapes, angles, spatial reasoning.			

□ Puzzle

Jeu	Sprint	Frontend PHP	Backend PHP
□ **Block Fit** *Tetris-like placement optimization in limited space.*	**W6**	`…/games/block_fit.php`	`… /block_fit /block_fit_backend.php`
□ **Maze Escape** *Pathfinding challenge with increasing complexity.*	**W6**	`…/games/maze_escape.php`	`… /maze_escape /maze_escape_backend.php`
□ **Jigsaw Puzzle** *Image reconstruction. Dynamic grid size.*	**W7**	`…/games/jigsaw_puzzle.php`	`… /jigsaw_puzzle /jigsaw_puzzle_backend.php`
□ **Bridge Builder** *Build structures with constraints. Physics lite intro.*	**W7**	`…/games/bridge_builder.php`	`… /bridge_builder /bridge_builder_backend.php`

□ Adventure

Jeu	Sprint	Frontend PHP	Backend PHP
□ **Mini Quest World** *Small missions with objectives and rewards.*	**W8**	`…/games/mini_quest.php`	`… /mini_quest /mini_quest_backend.php`
□ **Treasure Hunt** *Clue-based navigation and discovery.*	**W8**	`…/games/treasure_hunt.php`	`… /treasure_hunt /treasure_hunt_backend.php`
□ **Emotion Island Explorer** *Each zone = different emotion mechanics. YOPY flagship game.*	**W9**	`…/games/emotion_island.php`	`… /emotion_island /emotion_island_backend.php`

3. Architecture des signaux comportementaux (IA)

Tous les signaux sont collectés côté client par BehaviorCollector.js et stockés dans $SESSION['behavior'] — jamais persistés en base de données.

3.1 — Signaux collectés par jeu

Jeu	Catégorie	Signaux collectés (transmis en session PHP)
□ **Memory Matrix**	□	latency | error_rate | session_duration | retry_count
□ **Pattern Builder**	□	latency | error_rate | help_requests | completion_rate
□ **Mini Sudoku**	□	latency | error_rate | abandon_rate | session_duration
⚡ **Logic Gates Puzzle**	□	latency | error_rate | retry_count | help_requests
□ **Tap the Light**	⚡	reaction_time | miss_count | accuracy | session_duration
□ **Color Switch Rush**	⚡	reaction_time | error_rate | accuracy | frustration_index
□ **Avoid the Obstacles**	⚡	reaction_time | collision_rate | session_duration | abandon_rate
➕**Quick Math Duel**	□	latency | error_rate | speed_score | difficulty_progression
□ **Word Builder**	□	latency | error_rate | vocabulary_score | help_requests
□ **Science Quiz Adventure**	□	accuracy | latency | skip_rate | curiosity_index
□ **Shape & Geometry Game**	□	accuracy | latency | error_rate | completion_rate
□ **Block Fit**	□	move_count | session_duration | abandon_rate | retry_count
□ **Maze Escape**	□	pathfinding_efficiency | session_duration | backtrack_count | abandon_rate
□ **Jigsaw Puzzle**	□	move_count | session_duration | error_rate | completion_rate
□ **Bridge Builder**	□	attempt_count | session_duration | error_rate | persistence_score
□ **Mini Quest World**	□	decision_latency | exploration_rate | session_duration | curiosity_index
□ **Treasure Hunt**	□	clue_time | exploration_rate | hint_requests | completion_rate
□ **Emotion Island Explorer**	□	zone_selection | emotional_engagement | session_duration | revisit_rate

3.2 — Glossaire des 26 signaux comportementaux

Signal ID	Description
latency	Délai moyen entre l'affichage et la première action (ms)
error_rate	Taux erreurs / tentatives dans la session (%)
session_duration	Durée totale de la session (s)
retry_count	Nombre de recommencements sans abandon
help_requests	Nombre de clics sur l'aide ou les indices
completion_rate	Sessions terminées / sessions démarrées (%)
abandon_rate	Sessions quittées avant fin (%)
reaction_time	Temps de réaction moyen (ms) — jeux reflex
accuracy	Taux de bonnes réponses (%)
miss_count	Nombre de cibles manquées
frustration_index	Erreurs rapides successives → indice frustration calculé
speed_score	Score vitesse sur les défis chronométrés
difficulty_progression	Niveau de difficulté atteint en fin de session
vocabulary_score	Mots corrects formés
skip_rate	Questions ignorées / sautées (%)
curiosity_index	Zones ou sujets explorés spontanément
move_count	Nombre de mouvements réalisés (puzzles, blocs)
backtrack_count	Retours en arrière dans la navigation (labyrinthe)
pathfinding_efficiency	Ratio chemin optimal / chemin réel
attempt_count	Nombre de tentatives par niveau
persistence_score	Reprises après échec / abandons
collision_rate	Taux de collisions (runner)
decision_latency	Temps de décision aux choix narratifs (ms)
exploration_rate	Zones / indices explorés / total disponibles (%)
hint_requests	Nombre d'indices demandés
zone_selection	Zones d'émotion choisies en priorité
emotional_engagement	Temps passé dans chaque zone émotionnelle
revisit_rate	Retour sur des zones déjà visitées (%)

3.3 — Inférence émotionnelle : conditions de l'arbre

Le moteur IA parcourt un arbre conditionnel déterministe dont les nœuds testent les signaux agrégés de la session. Les quatre états possibles et leurs conditions :

ETAT : ENGAGÉ	
Conditions	- latency < seuil_moyen - error_rate < 30% - completion_rate > 70% - session_duration dans plage normale

ETAT : FRUSTRÉ	
Conditions	- latency > 2x seuil_moyen - error_rate > 60% - retry_count > 5 - abandon_rate élevé

ETAT : CONFUS	
Conditions	- latency élevée + error_rate moyen - help_requests > seuil - backtrack_count élevé (maze)

ETAT : SATISFAIT	
Conditions	- completion_rate > 85% - score élevé - session_duration normale - retry_count faible

3.4 — Interface JavaScript BehaviorCollector.js

```
// public/js/games/BehaviorCollector.js
// Collecte les signaux comportementaux et les envoie en session PHP

class BehaviorCollector {
  constructor(gameId) {
    this.gameId     = gameId;
    this.signals    = {};        // accumulateur de session
    this.startTime  = Date.now();
  }

  record(signalName, value) {
    // value peut être un compteur, une durée, un ratio
    if (!this.signals[signalName]) this.signals[signalName] = [];
    this.signals[signalName].push({ value, ts: Date.now() });
  }

  aggregate() {
    // Calcule latence_moy, error_rate, session_duration, etc.
    const agg = {};
    if (this.signals.action_times?.length) {
      const times = this.signals.action_times.map(e=>e.value);
      agg.latency = times.reduce((a,b)=>a+b,0)/times.length;
    }
    if (this.signals.errors && this.signals.attempts) {
      agg.error_rate = this.signals.errors.length /
                       this.signals.attempts.length;
    }
```

```
    agg.session_duration = (Date.now() - this.startTime) / 1000;
    return agg;
  }

  async flush() {
    // Envoie les signaux au backend → stockés en $_SESSION['behavior']
    const payload = {
      game_id:  this.gameId,
      signals:  this.aggregate(),
      raw_count: Object.keys(this.signals).length
    };
    await fetch('/game/session/behavior', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    });
  }
}
```

3.5 — Backend session handler PHP (partagé)

```
// games/shared/session_behavior.php
// Reçoit les signaux de BehaviorCollector.js et les merge en session

<?php
session_start();
header('Content-Type: application/json');

$data = json_decode(file_get_contents('php://input'), true);
if (!$data || !isset($data['game_id'], $data['signals'])) {
    http_response_code(422);
    echo json_encode(['status'=>'error','message'=>'Invalid payload']);
    exit;
}

// Merge dans $_SESSION['behavior'] — JAMAIS en base de données
if (!isset($_SESSION['behavior'])) {
    $_SESSION['behavior'] = [];
}
$gid = htmlspecialchars($data['game_id']);
$_SESSION['behavior'][$gid] = array_merge(
    $_SESSION['behavior'][$gid] ?? [],
    $data['signals']
);

echo json_encode(['status'=>'success','stored'=>count($data['signals'])]);
```

4. Anatomie des fichiers — Templates

4.1 — Frontend PHP : anatomie d'un jeu (template)

Chaque views/child/games/{game}.php suit cette structure :

```
<?php
// views/child/games/{game_id}.php
```

```
session_start();
if (!isset($_SESSION['user_id'])) {
    header('Location: ../../auth/login.php'); exit;
}
$nickname = htmlspecialchars($_SESSION['nickname'] ?? 'Explorer');
?>
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>YOPY — {Game Name}</title>
  <!-- YOPY shared CSS -->
  <link rel="stylesheet" href="../../../public/css/game.css">
  <!-- Google Fonts -->
  <link href="https://fonts.googleapis.com/..." rel="stylesheet">
</head>
<body>

  <!-- 1. TOP BAR (shared) -->
  <?php include '../../../views/layouts/topbar_child.php'; ?>

  <!-- 2. GAME STAGE -->
  <div id="gameStage">
    <!-- game UI rendered by JS -->
  </div>

  <!-- 3. HUD (score, timer, lives) -->
  <div id="gameHUD">
    <span id="scoreDisplay">0</span>
    <span id="timerDisplay"></span>
  </div>

  <!-- 4. RESULT OVERLAY -->
  <div id="resultOverlay" class="hidden">
    <!-- Score + emotion feedback -->
  </div>

  <!-- 5. SCRIPTS -->
  <script src="../../../public/js/games/BehaviorCollector.js"></script>
  <script src="../../../public/js/games/GameEngine.js"></script>
  <script src="../../../public/js/games/{game_id}.js"></script>
  <script>
    const GAME_ID      = '{game_id}';
    const SESSION_ID   = '<?= $_SESSION["session_id"] ?? "" ?>';
    const BUDDY_EMOTE  = '<?= $_SESSION["chosen_character"]["id"] ?? "joy" ?>';
    const collector    = new BehaviorCollector(GAME_ID);
    const engine       = new {GameClass}(GAME_ID, collector);
    engine.init();
  </script>

</body></html>
```

4.2 — Backend PHP : anatomie (template)

```
<?php
// games/{game_id}/{game_id}_backend.php
// Handles: session/start | session/score | session/behavior
session_start();
header('Content-Type: application/json');
```

```php
require_once '../../config/database.php';
require_once '../../services/GameService.php';
require_once '../../services/ScoreService.php';

$action = $_GET['action'] ?? '';
$pdo    = Database::connect();

match($action) {
  'start' => handleStart($pdo),
  'score' => handleScore($pdo),
  default => http_response_code(404)
};

// ── START : crée une game_session en BD ──────────────────────────────
function handleStart(PDO $pdo): void {
  $childId  = (int)($_SESSION['child_id'] ?? 0);
  $gameId   = GameService::getIdBySlug($pdo, '{game_id}');
  $difficulty = $_POST['difficulty'] ?? 'easy';

  // Validation (BR-08 : difficulté figée en cours de jeu)
  $sessionId = GameService::createSession($pdo, $childId, $gameId, $difficulty);
  $_SESSION['current_game_session'] = $sessionId;
  echo json_encode(['status'=>'success','session_id'=>$sessionId]);
}

// ── SCORE : valide et persiste le score ──────────────────────────────
function handleScore(PDO $pdo): void {
  $data       = json_decode(file_get_contents('php://input'), true);
  $sessionId  = (int)($data['session_id'] ?? 0);
  $points     = (int)($data['points'] ?? 0);
  $duration   = (int)($data['completion_time'] ?? 0);

  // BR-06 : rejet si hors plage
  if ($points < 0 || $points > 10000) {
    http_response_code(422);
    echo json_encode(['status'=>'error','message'=>'Score out of range']);
    return;
  }
  // BR-05 : unique par session
  ScoreService::save($pdo, $sessionId, $points, $duration);
  echo json_encode(['status'=>'success']);
}
```

5. Plan de développement — 10 Sprints

Chaque sprint dure 1 semaine. Les jeux sont développés dans un ordre progressif : infrastructure d'abord, jeu phare (Emotion Island) en dernier.

Sprint 1 — Infrastructure + Cognitive (1)	
Pose la fondation : collecte de signaux + premiers jeux cognitifs	
Livrables	[+] GameEngine.js (client state machine) [+] BehaviorCollector.js (signal recording) [+] GameSession PHP handler [+] Memory Matrix

	(frontend + backend) [+] Pattern Builder (frontend + backend)

Sprint 2 — Cognitive (2)	*Finit la catégorie cognitive*
Livrables	[+] Mini Sudoku (frontend + backend) [+] Logic Gates Puzzle (frontend + backend) [+] ScoreService + validation server [+] Tests unitaires jeux cognitifs

Sprint 3 — Reflex & Reaction (1/2)	*Jeux temps réel — collecte reaction_time*
Livrables	[+] Tap the Light (frontend + backend) [+] Color Switch Rush (frontend + backend) [+] ReactionTimer.js (précision ms) [+] Tests catégorie reflex

Sprint 4 — Reflex (fin) + Educational (1)	*Runner + maths adaptatives*
Livrables	[+] Avoid the Obstacles (frontend + backend) [+] Quick Math Duel (frontend + backend) [+] DifficultyEngine.js (adaptive)

Sprint 5 — Educational (fin)	*Valide le sous-système éducatif complet*
Livrables	[+] Word Builder (frontend + backend + dictionary validation) [+] Science Quiz Adventure (frontend + backend) [+] Shape & Geometry (frontend + backend)

Sprint 6 — Puzzle (1/2)	*Jeux spatiaux et raisonnement*
Livrables	[+] Block Fit / Tetris-like (frontend + backend) [+] Maze Escape (frontend + backend) [+] PathfindingUtils.js

Sprint 7 — Puzzle (fin)	*Termine la catégorie puzzle*
Livrables	[+] Jigsaw Puzzle (frontend + backend) [+] Bridge Builder / Physics Lite (frontend + backend) [+] Tests catégorie puzzle

Sprint 8 — Adventure (1/2)	*Moteur narratif et exploration*

Livrables	[+] Mini Quest World (frontend + backend) [+] Treasure Hunt (frontend + backend) [+] NarrativeEngine.js (décisions)

Sprint 9 — Emotion Island (flagship) | *Jeu phare aligné sur le thème YOPY*	
Livrables	[+] Emotion Island Explorer (frontend + backend) [+] Zone émotionnelle mappée sur les 6 Emotion Buddies [+] Integration complete BehaviorCollector → session → dashboard

Sprint 10 — IA + Intégration finale | *Active le moteur IA sur l'ensemble des jeux*	
Livrables	[+] BehaviorAnalyzer.php (arbre conditionnel sur données multi-jeux) [+] EmotionState.php complet [+] Tests de bout-en-bout signal → état → dashboard [+] Rapport d'état émotionnel dans dashboard parent

6. Script de création automatique des fichiers

Le script bash suivant crée tous les fichiers et dossiers en une commande :

```
#!/bin/bash
# setup_games.sh — Crée la structure de tous les jeux YOPY
ROOT="PRO-PFE-YOPY/yopy-platform"

GAMES=(
  memory_matrix pattern_builder mini_sudoku logic_gates
  tap_the_light color_switch avoid_obstacles
  quick_math word_builder science_quiz shape_geometry
  block_fit maze_escape jigsaw_puzzle bridge_builder
  mini_quest treasure_hunt emotion_island
)

for game in "${GAMES[@]}"; do
  # Frontend
  touch "$ROOT/views/child/games/${game}.php"
  # Backend
  mkdir -p "$ROOT/games/${game}"
  touch "$ROOT/games/${game}/${game}_backend.php"
  # JS engine
  touch "$ROOT/public/js/games/${game}.js"
done

# Shared JS files
touch "$ROOT/public/js/games/BehaviorCollector.js"
touch "$ROOT/public/js/games/GameEngine.js"
echo "✓ Structure créée : ${#GAMES[@]} jeux × 3 fichiers"
```

7. Jeu Phare — Emotion Island Explorer

Ce jeu est directement aligné sur le thème YOPY. Chaque zone représente une émotion et génère des signaux spécifiques pour le moteur IA.

Zone	Mécanique de jeu	Signaux collectés
Zone Joie	Défi de rapidité et de récompenses — bonus en cascade	`reaction_time` `speed_score` `accuracy` `emotional_engagement`
Zone Tristesse	Puzzles de reconnexion narrative — pièces à replacer	`session_duration` `completion_rate` `help_requests` `emotional_engagement`
Zone Colère	Défi de précision sous pression — cibles mobiles	`frustration_index` `error_rate` `reaction_time` `emotional_engagement`
Zone Dégoût	Tri et classification — choisir les bons éléments	`accuracy` `decision_latency` `error_rate` `emotional_engagement`
Zone Peur	Progression progressive dans l'obscurité — confiance croissante	`abandon_rate` `retry_count` `session_duration` `emotional_engagement`
Zone Anxiété	Planification pas-à-pas — décomposer une tâche complexe	`decision_latency` `help_requests` `completion_rate` `emotional_engagement`

8. Récapitulatif des métriques du projet

Dimension	Valeur
Jeux total	18 jeux
Catégories	5 (Cognitif, Reflex, Éducatif, Puzzle, Aventure)
Fichiers frontend PHP	18 (views/child/games/{game}.php)
Fichiers backend PHP	18 (games/{game}/{game}_backend.php)
Fichiers JS engines	18 + 2 partagés (GameEngine.js, BehaviorCollector.js)
Signaux comportementaux	26 types, collectés en $_SESSION uniquement
États émotionnels	4 (Engagé, Frustré, Confus, Satisfait)
Sprints	10 semaines (phases S1→S9 + intégration IA S10)
Script setup	setup_games.sh — génère 56 fichiers en une commande
Jeu phare	Emotion Island Explorer (Sprint 9) — 6 zones × 1 émotion